Routing, Not Reasoning: A Fail-Closed LLM Agent for Natural-Language Netlist Transformation

Anonymous Authors
Paper under double-blind review

***Abstract*—LLM agents that edit gate-level netlists must interpret free-form requests without corrupting the design, often with small, frozen models. We present a fail-closed agent in which the LLM only maps each request to one intent from a closed catalog of operations; a deterministic engine executes it, and a mutation commits only after equivalence and structural checks pass. On a 1,420-request paraphrase stress set for the ICCAD 2026 CAD Contest, the catalog raises GPT-4o-mini routing accuracy from 61.4% to 88.4%, beyond supervised classifiers (73.3%). Fault injection found no unsafe commit after one repair, yet exposed equivalent but non-conforming edits.**

I. Introduction

Natural-language (NL) interfaces are entering EDA flows, from design question answering to tool-script generation and RTL repair [1]–[3]. Applied to gate-level netlist manipulation, such an interface must meet two requirements at once. First, correctness is not a quality score: a netlist whose function has been silently altered is unusable downstream, which is why production flows gate changes behind equivalence checking. Second, the design state persists: requests arrive in sequence and operate on the netlist as earlier requests left it, so an early error can shape every answer that follows. These requirements often come with limited model capability. Organizations that treat designs as confidential IP may prefer small, locally deployable models [4]; benchmarks such as the ICCAD 2026 CAD Contest Problem A [5] mandate frozen mini-class models; and few teams maintain a fine-tuning pipeline.

When the LLM itself generates the executable artifact—a tool script, an edited netlist, or RTL code—its errors have a direct path into the design. Post-hoc checks catch many of them, but the failure surface is hard to bound, and a smaller model raises the error rate the checks must absorb.

We take the opposite approach: the LLM routes; it does not rewrite. Its only output is one intent object per request, drawn from a closed catalog of about 80 predefined operations with typed parameters. A deterministic engine performs every circuit operation. Parameters that name nets or gates are grounded against the current netlist before execution, and each mutation runs on a snapshot that is committed only after a proof of functional equivalence, obtained through a register-cut projection, and checks of the operation's structural constraints; otherwise the snapshot is restored.

This architecture bounds what the LLM can damage, but not what it can misunderstand. Under explicitly stated assumptions, every committed netlist is functionally equivalent to its predecessor; a mis-routed request can still commit a valid but unwanted edit, and that edit persists into later requests. We therefore evaluate both sides: how accurately small frozen models route requests, and which faults the guards prevent and which they cannot.

Our contributions are:

- **Architecture.** A constrained interface for NL netlist manipulation that separates LLM intent interpretation from deterministic execution and proof-gated commit, with an explicit statement of what it guarantees (functional integrity, under stated assumptions) and what it does not (request conformance) (Section III).
- **Routing evaluation.** A controlled study on 1,420 paraphrased requests that compares a bare intent list, a full catalog, retrieval configurations, model choices, and supervised baselines, and analyzes the near-synonym operations behind most residual errors (Sections IV-B–IV-C).
- **Validation and limits.** Fault injection and the official contest evaluation, quantifying prevented violations, residual semantic drift, and one guard-bypassing operation that the experiment exposed and that we repaired (Sections IV-D–IV-E).

II. Problem Setting and Related Work

Setting. A session loads one flat gate-level Verilog netlist—two-input primitive gates, inverters, buffers, and D flip-flops with asynchronous control—and then receives free-form NL requests, one per line, answered in lock-step [5]. Requests analyze (depth, cones, paths, equivalence), edit (buffer insertion, basis conversion, renaming), or optimize (depth, gate count) the current netlist. Hard requirements—functional equivalence where requested, stated structural bounds, gate-basis restrictions—are all-or-nothing: violating any one zeroes the testcase.

LLM agents for EDA. Script-generating agents emit tool scripts for EDA flows and check them before execution [1], [6], [7]; flow-tuning agents search a bounded parameter space [8], [9]; FluxEDA keeps backend instances alive so that later steps observe earlier state [10]. RTL-oriented work generates or repairs code against compilers and testbenches [2], [3], [11], and recent benchmarks target netlist manipulation and tool calling [12], [13]. These systems differ in how much execution freedom the LLM holds; in ours it holds none beyond selecting one predefined operation and its parameters on a persistent gate-level netlist.

Structured tool calling. Schema-constrained function calling is well established [14]–[16], typically retrieving tool

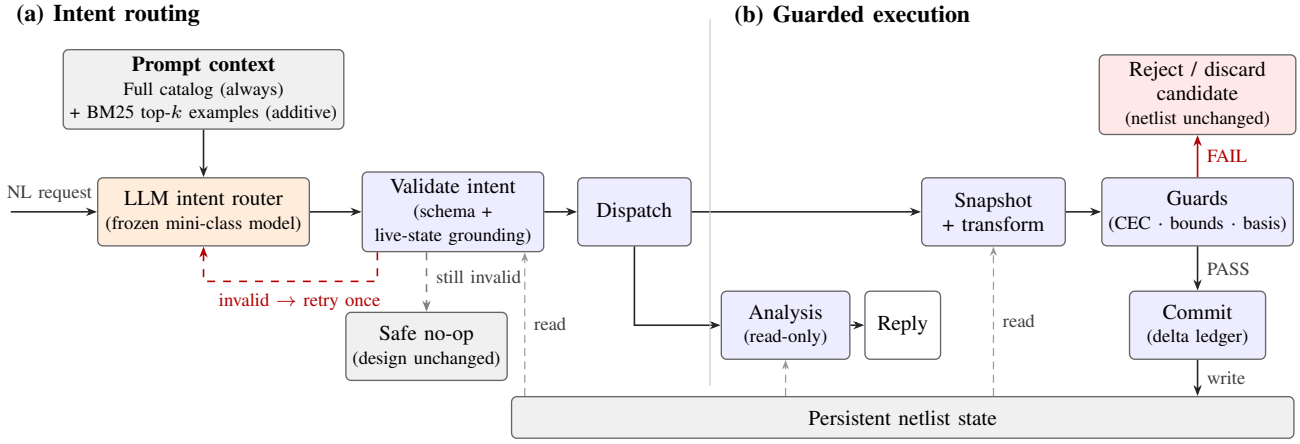


Fig. 1. Overview of the fail-closed netlist agent. (a) The LLM maps each natural-language request to an intent, using the complete operation catalog and additional BM25-retrieved demonstrations. Schema validation and grounding against the current netlist allow one corrective retry; a request that is still invalid yields a safe no-op. (b) Read-only queries execute directly, whereas mutations operate on a snapshot and commit only after the equivalence and structural checks pass; rejected candidates leave the persistent netlist unchanged. Dashed gray arrows: reads of the current netlist.

documentation into the prompt. We instead retrieve labeled example requests, keep the full catalog in every prompt, and validate name-typed parameters against a design state that earlier requests have modified.

Verified and checked execution. LLM-Modulo argues for LLM generation behind external verifiers [17]; obligation and transaction checks constrain agent actions outside EDA [18]–[20]. Concurrent work on the same benchmark shows that a run passing every equivalence check can still contain incorrect answers and violated constraints, and adds request-derived completion checks [21]. That work checks conformance, whereas ours bounds functional damage; the two are complementary. Our contribution is therefore not a new verification primitive but a commit protocol for persistent sequential netlists, state-aware parameter grounding, and a measured account of what the combination prevents.

III. Fail-Closed Agent Design

A. Closed-World Intent Interface

The catalog defines about 80 intents covering analysis (counts, depth, cones, paths, connectivity, functional properties), transformation (basis remapping, buffer insertion, cleanup, renaming), and optimization (depth, gate count). Each intent declares required and optional parameters with types; enumerated parameters such as the gate basis have closed value sets. The catalog text—a description of each intent plus disambiguation entries for operations that users phrase similarly—is part of every prompt as a fixed, cacheable prefix of about 12.8k tokens. Retrieval only adds examples and never removes catalog entries, so a retrieval miss cannot put the correct operation out of the model's reach.

The operation space is closed; the parameter space is not. Which net, cone, bound, or basis an operation applies to is decided per request. A request that maps to no catalog operation yields a safe no-op that reports the design unchanged. Extending the catalog means adding a handler, a schema, and a disambiguation entry—an auditable engineering change rather than model retraining.

B. Routing with Live-State Grounding

Routing proceeds in four stages (Fig. 1a). (1) *Translation:* the LLM receives the catalog and the 50 most similar labeled requests, retrieved by BM25 from a bank of 940 labeled requests from contest-released testcases and our development sessions, and emits one JSON intent object. (2) *Schema validation:* an unknown intent, a missing parameter, or an illegal enumerated value is rejected, and the rejection reason is returned to the LLM for one corrective retry. (3) *Name grounding:* every parameter that must name a net, gate, or flip-flop is checked against the current netlist, which earlier requests may have modified; a nonexistent name triggers re-translation with the error message. (4) *Dispatch* to a deterministic handler: analysis handlers run graph algorithms on the netlist IR, and mutation handlers enter the commit protocol of Section III-C. Translations are cached within a session, and if every stage fails the agent returns the safe no-op.

C. Guarded Execution and Commit Protocol

A mutation handler transforms a snapshot of the netlist (Fig. 1b), and three checks decide whether the snapshot replaces the current design.

1) **Functional equivalence.** A register-cut projection turns the sequential netlist into a combinational one: each flip-flop's D, CK, RN, and SN pins become pseudo-outputs and its Q pin a pseudo-input. Flip-flops correspond by instance identity, and a change to the register set is rejected outright. Because every flip-flop input is observed, combinational equivalence of the projections implies sequential equivalence for the testbed's flip-flop model, including logic on clock and asynchronous pins. ABC's `cec` decides equivalence [22], with a yosys check as a fallback.

2) **Structural bounds.** Bounds carried by the operation, such as a maximum fanout, are recomputed on the result.
3) **Basis purity.** If the operation restricts gate types, the gate-type set of the result is checked.

The acceptance rule is fail-closed: the snapshot is committed only when every check passes and equivalence is proved. A disproof, a timeout, or an inconclusive result—for instance on a malformed netlist—restores the snapshot. Candidates produced by the optimizer, including precomputed candidates that serve as search seeds, obey the same rule. Committed mutations are recorded in a delta ledger that later requests ("how many buffers were added?") query.

D. Scope of the Guarantee

We distinguish three properties of a committed edit: *functional integrity* (the netlist computes the same function), *structural validity* (it meets the bounds and basis of the routed operation), and *request conformance* (the routed operation is what the user asked for). Every mutation intent in the catalog is function-preserving by specification; a request that would change function (e.g., inserting a gating cell) has no catalog operation and cannot be executed as requested. The architecture targets the first two properties, under four assumptions:

- **(A1) Complete mediation.** Every operation that mutates the netlist, including renaming, executes through the commit protocol.
- **(A2) Sound checking.** A proof of equivalence reported by the checker is correct for the projected circuits.
- **(A3) Faithful projection.** The register-cut model matches the flip-flop semantics of the netlist.
- **(A4) Correct structural checks.** The bound and basis checks accept a netlist only if it meets the bounds and basis carried by the operation.

Proposition 1: Under A1–A4, every netlist the agent commits is functionally equivalent to the netlist it replaces, and hence to the most recently loaded design, and meets the bounds and basis of the routed operation. An inconclusive check never produces a commit.

Proof sketch: By A1, the current netlist changes only when a snapshot is committed, and a commit requires a proof of equivalence of the register-cut projections and passing structural checks. By A2 the proof is correct for the projections, by A3 projection equivalence implies sequential equivalence, and by A4 the structural checks are correct. Equivalence is transitive, so induction over the sequence of commits extends it to the loaded design. A disproof, a timeout, or an inconclusive result restores the snapshot by the acceptance rule. ■

The proposition says nothing about request conformance: a mis-routed request can commit an equivalent but unwanted edit, and because the design persists, that edit changes what later requests observe—for example, a gate count or a net name that a later question depends on. The guards also enforce only the requirements carried by the current operation, not constraints established by earlier requests. A1 is the assumption most exposed to implementation error, and Section IV-D tests it directly.

IV. Evaluation

A. Setup

Requests. We evaluate routing on a stress set of 1,420 requests that our team wrote during the contest by paraphrasing the official request patterns: 71 operations with 20 paraphrases each, of which 1,065 use deliberately indirect or unusual wording and 355 use standard phrasing. The set provides ground-truth intents and parameters but no netlists, so we score the translation output directly. The retrieval bank of 940 labeled requests (703 from contest-released testcases, 237 from author-constructed development sessions) is disjoint from the stress set. A second, high-recall bank of 2,130 author-written paraphrases contains paraphrases of the same operations as each test request; we use it with leave-one-testcase-out (LOCO) exclusion and only as an upper bound, because it shares the stress set's distribution. Since the catalog's disambiguation entries were refined during the contest while inspecting errors on this set, accuracies on it may be optimistic.

Sessions. Fault injection runs on 40 development sessions (24 from the contest's public release, 16 constructed by the authors) and the official evaluation on the contest's final round (Table II). Final-round designs reach 180k gates and 33k flip-flops, and a session chains up to 15 requests on one persistent netlist.

Metric and models. The primary metric is intent accuracy: whether the selected operation is correct. We use the two contest-mandated models, GPT-4o-mini and Claude Haiku 4.5, at temperature 0.2, and repeat the GPT-4o-mini headline settings five times (N=5). Evaluation rounds run strictly one at a time, and any round with a degraded API call is discarded, because rate limiting can silently turn LLM calls into no-op responses.

B. Routing Accuracy

Catalog information is the largest lever. Table I lists all configurations. A bare list of intent names yields 61.4% with GPT-4o-mini; the full catalog—descriptions, parameter schemas, and disambiguation entries together—raises it to 88.4% (+27.0 points). The bare condition keeps the schema-feedback retry, whose fixed message mentions class-level depth intents, and contains no other catalog text. The gap between the two models is 17.3 points with the bare list but 8.0 points with the full catalog, so the catalog helps the weaker model more.

Supervised baselines underperform. With every labeled request available, the best classifier reaches 73.3% (TF-IDF features with a linear SVM), and frozen sentence embeddings reach 72.8%. Classifiers and LLMs are scored on the same operation-level metric; the LLM additionally fills in parameters (84.6% parameter accuracy for GPT-4o-mini with the catalog, 94.1% for Claude Haiku 4.5 with retrieval), which the classifiers do not attempt. With only the production bank, three

TABLE I

INTENT ROUTING ACCURACY ON THE AUTHOR-WRITTEN 1,420-REQUEST PARAPHRASE STRESS SET. VALUES ARE PERCENTAGES; LLM VALUES ARE MEANS OVER N RUNS. EXACT OBJ. REQUIRES THE INTENT AND ALL PARAMETERS TO BE CORRECT.

Router	Prompt / configuration	Data source	Intent (%)	Exact obj. (%)	N
Non-LLM baselines					
BM25 NN	—	Production bank	47.4	—	—
SBERT + kNN	—	All labels (LOCO)	72.8	—	—
TF-IDF + SVM	—	All labels (LOCO)	73.3	—	—
GPT-4o-mini ablations					
GPT-4o-mini	Bare intent list	None	61.4	55.4	3
GPT-4o-mini	Full catalog	None	88.4	79.5	5
GPT-4o-mini	Full catalog + BM25	Production bank	88.9	82.0	5
GPT-4o-mini	Full catalog + BM25	High-recall bank†	95.4	91.6	2
Claude Haiku 4.5 ablations					
Claude Haiku 4.5	Bare intent list	None	78.7	72.3	1
Claude Haiku 4.5	Full catalog	None	96.4	91.4	3
Claude Haiku 4.5	Full catalog + BM25	Production bank	**97.4**	**92.7**	1

†The high-recall bank uses LOCO exclusion and shares the stress set's distribution; it is an optimistic upper bound. LOCO: leave-one-testcase-out. For non-LLM methods, "Data source" denotes training or reference data; for LLMs, retrieved demonstrations (k=50). Min–max over runs: GPT-4o-mini bare 61.1–61.8, full catalog 88.3–88.7, production bank 88.6–89.2, high-recall bank 95.1–95.6; Claude Haiku 4.5 full catalog 96.3–96.5.

TABLE II

SESSIONS USED FOR FAULT INJECTION (DEVELOPMENT: 24 CONTEST-RELEASED, 16 AUTHOR-CONSTRUCTED) AND THE OFFICIAL EVALUATION (CONTEST FINAL ROUND). REQUEST CATEGORIES FOLLOW THE REQUIREMENT CLASSIFIER OF OUR EVALUATION HARNESS.

	Development	Final round
Sessions / requests	40 / 459	100 / 674
Analysis / edit / optimize	390/57/12	590/38/46
Requests per session (med./max)	9/20	7/15
Gates (min/med./max)	66/3,536/96,064	32/3,355/180,047
Flip-flops (med./max)	27/16,050	345/33,232

operations have no examples and are unreachable for nearest-neighbor routing, whereas the LLM reaches them through the catalog.

Retrieval adds little with the production bank. Retrieving k=50 examples from the production bank adds 0.5 points for GPT-4o-mini and 1.0 point for Claude Haiku 4.5 over the catalog alone, and retrieval depth barely matters: for GPT-4o-mini, k from 5 to 100 spans 88.1–89.4% without a monotone trend, while label recall rises from 60% to 86%. The catalog thus carries most of the routing accuracy: Claude Haiku 4.5 reaches 96.4% with no retrieval at all, even though the production bank's label recall at k=50 is only 82.1%. Retrieval pays off only when the examples closely match the requests: the high-recall bank adds 6.4 points for GPT-4o-mini, an upper bound because it shares the stress set's distribution, and an effect comparable to switching models (+8.0 points with the catalog alone).

C. Failure Analysis

Moving GPT-4o-mini from the production to the high-recall bank removes 84% of give-ups (44 to 7) but only 43% of wrong-operation picks (110 to 63): good examples mainly help the model recognize unfamiliar phrasing. What remains

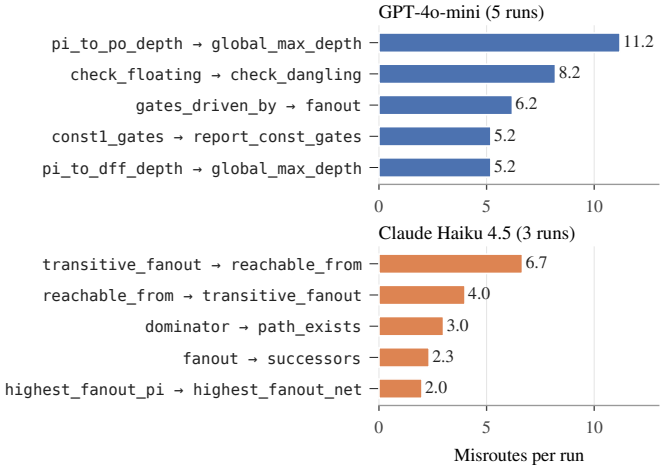


Fig. 2. Most frequent intent confusions with the full catalog and no retrieval (misroutes per run on the 1,420-request stress set; give-ups excluded). The five pairs shown account for 31% (GPT-4o-mini) and 53% (Claude Haiku 4.5) of each model's misroutes.

concentrates on a few pairs of semantically similar operations (Fig. 2). With the full catalog, the five most frequent pairs account for 31% of GPT-4o-mini's misroutes and 53% of Claude Haiku 4.5's, and the two models confuse different families. GPT-4o-mini mostly mistakes a scoped depth query (from primary inputs to primary outputs, or to flip-flops) for the global maximum depth, and floating for dangling nets. Claude Haiku 4.5 mostly swaps transitive fanout and reachability, in both directions, which points to genuine ambiguity in the requests rather than to a consistent model error. The supervised baselines show several of the same confusions, such as dominators versus path existence.

The concurrent agent of [21], built on a different architecture, reports the same kind of scope error on this benchmark

TABLE III
FAULT-INJECTION OUTCOMES. ROUTING FAULTS CORRUPT THE ROUTER'S OUTPUT AT DISPATCH; ENGINE FAULTS CORRUPT A TRANSFORMATION INSIDE THE ENGINE. CLASSES ARE MUTUALLY EXCLUSIVE.

		Routing (21)		Engine (3)
Outcome	Meaning	before fix	after fix	
Rejected	refused before execution	2	5	0
Rolled back	equivalence disproved	0	0	3
Benign	met every hard requirement	9	9	0
Drift	equivalent but non-conforming	7	7	0
Unsafe	non-equivalent netlist committed	3	0	0

(a pairwise depth of 10 reported as the whole-design depth of 53). On this request distribution, near-synonym discrimination is a major source of residual routing errors.

D. Fault Injection

To test A1 and the guards, we use two fault models. *Routing faults* corrupt the router's output at the dispatch layer, simulating worst-case LLM behavior: numeric bounds off by one in either direction, a wrong but implemented basis or transformation, a basis outside the closed world, a target redirected to a different existing net, and a swap to a different intent. *Engine faults* instead inject a function-changing rewire inside a transformation, standing in for an implementation bug. We run 24 injections (21 routing, 3 engine) over six transformation-heavy development sessions (the six uncorrupted runs pass all checks) and classify each final netlist against the hard requirements of the original request (Table III).

The two fault models are stopped by different mechanisms. Every rollback came from an engine fault: the equivalence check caught all three function-changing rewires. No routing fault produced a function-changing edit through the commit protocol, because every catalog operation preserves function; routing faults were either refused by the closed world or yielded an equivalent netlist. Only rejections and rollbacks are detections; the nine benign cases passed because the corrupted edit happened to remain acceptable. The three unsafe commits shared one cause, a violation of A1: renaming was treated as cosmetic and bypassed the commit protocol, so renaming a net onto an existing name merged two drivers while the agent reported success. In one of these cases ABC returned an inconclusive result on the double-driven net, which an evaluation harness that treats inconclusive results as passing would have accepted. After we added a check that rejects renames onto existing names, no injection produced an unsafe commit, and the 40 development sessions pass all 84 of their hard-requirement checks. The seven drift cases mark the conformance boundary of Section III-D: the guards enforced the corrupted operation's contract rather than the user's sentence, and the response claimed success. Detecting them requires request-level completion checks such as those of [21].

Because the design persists, a single corrupted request also changes later answers: in 13 of the 24 runs, at least one later response differs from the uncorrupted run (27 of 173 later responses), including 5 of the 7 drift runs, 4 of 9 benign runs, 2 of 3 rolled-back runs, and 2 of 5 rejected runs. Fig. 3 traces one testcase. Divergence also follows the safe outcomes, because a refused or rolled-back request leaves the design in a state the user did not intend; the later answers are then correct for the actual design, whereas after drift the agent had reported success. A changed answer is therefore not necessarily a wrong one.

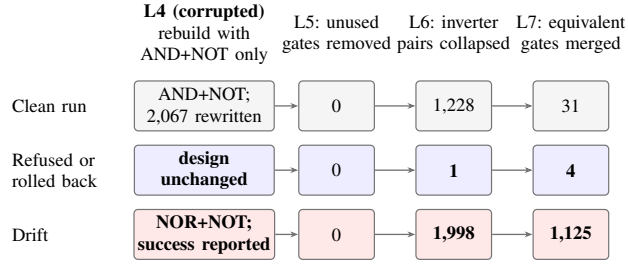


Fig. 3. Measured persistent-state divergence in development session test29 (selected requests; cells give the count each response reports, bold where it differs from the clean run). The routing of L4 is corrupted. Refusing it (basis outside the closed world) or rolling it back (engine fault) leaves the design unchanged; mis-routing it to an equivalent NOR+NOT remap commits an unwanted edit with a success message. Every later answer is correct for its run's actual design, yet differs from the clean run.

TABLE IV
OPTIMIZATION PERFORMANCE ON THE 46 FINAL-ROUND OPTIMIZATION REQUESTS. NORMALIZED SCORE: MEAN REQUEST SCORE × 100%.

Configuration	Normalized score	Compute
Official evaluation	94.27%	1× (300 s per request)
Re-run, same configuration	94.60%	1×
Best of up to four runs†	95.62%	up to 4×

†Post-contest upper bound; includes an earlier engine version.

E. Official Evaluation and Post-Contest Analysis

In the contest's final round (100 testcases and 674 requests, unseen during development), the submitted configuration met every hard requirement and scored 671.366 of 674 under official scoring. All losses came from optimization requests, which are scored by their cost relative to the best cost among all entries. Table IV compares the official result with two post-contest re-evaluations of the same 46 optimization requests.

A single re-run under the same per-request budget improves 15 requests and worsens 4, for a net gain of 0.32 percentage points: the optimizer's search is budget-bound and stochastic, and one run is far from its best achievable result. Taking the best of up to four runs, at up to four times the compute, adds 1.34 percentage points; all 17 improved netlists were re-verified independently with ABC, and four of them beat the best cost among all contest entries. These figures are post-contest upper bounds, not a comparison under the official protocol. Three requests resist re-sampling entirely (3.1 percentage points) and call for algorithmic improvement.

V. Limitations and Conclusion

Limitations. We evaluate on one benchmark. The routing stress set was written by the authors and the catalog refined against it, so its accuracies may be optimistic; the official final round provides an out-of-sample, end-to-end check. The cloud mini-class models stand in for the small, locally deployable models that motivate this work, and direct validation on open-weight models remains future work. Some settings were run once ($N{=}1$). The functional guarantee rests on A1–A4 and on a correct implementation of them, and drift—valid but non-conforming edits—is outside its scope.

Conclusion. Restricting the LLM to selecting predefined operations, and committing their results only on proof, bounds the functional damage that an LLM error can cause in persistent netlist manipulation, under assumptions that can be stated and tested; fault injection found and closed a gap in the assumption most exposed to implementation error. What the architecture does not bound—whether the executed operation is the one the user meant—remains, and our routing study locates most of it in near-synonym operations. For agents that edit an existing design, safe execution and request conformance are separate problems, and each needs explicit treatment.

Acknowledgment

AI coding assistants were used for implementation, data post-processing, and language editing; all AI-assisted output was reviewed and revised by the authors, who take full responsibility for the content.

REFERENCES

[1] H. Wu, Z. He, X. Zhang, X. Yao, S. Zheng, H. Zheng, and B. Yu, "ChatEDA: A large language model powered autonomous agent for EDA," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 43, no. 10, pp. 3184–3197, 2024.

[2] Y. Tsai, M. Liu, and H. Ren, "RTLFixer: Automatically fixing RTL syntax errors with large language models," in *Proc. DAC*, 2024.

[3] M. Liu, N. Pinckney, B. Khailany, and H. Ren, "VerilogEval: Evaluating large language models for Verilog code generation," in *Proc. ICCAD*, 2023.

[4] M. Liu *et al.*, "ChipNeMo: Domain-adapted LLMs for chip design," arXiv:2311.00176, 2023.

[5] C.-H. Chou, H.-C. Chiu, C.-J. Hsu, and Y. Liu, "ICCAD Contest Problem A: LLM-assisted netlist exploration and transformation," problem specification and Q&A, 2026.

[6] G. Pasandi *et al.*, "JARVIS: A multi-agent code assistant for high-quality EDA script generation," arXiv:2505.14978, 2025.

[7] B.-Y. Wu, U. Sharma, A. Rovinski, and V. A. Chhabria, "OpenROAD Agent: An intelligent self-correcting script generator for OpenROAD," in *Proc. IEEE ICLAD*, 2025, pp. 16–22.

[8] A. Ghose, A. B. Kahng, S. Kundu, and Z. Wang, "ORFS-agent: Tool-using agents for chip design optimization," in *Proc. ACM/IEEE MLCAD*, 2025, pp. 1–13.

[9] K. Li, S. Yao, S. Zheng, and L. Wang, "StateTune: Transforming LLM-assisted EDA flow tuning into a stateful, closed-loop process," in *Proc. IEEE/ACM ICCAD*, 2026.

[10] Z. Chen, Z. Song, Y. Li, Q. Sun, and C. Zhuo, "FluxEDA: A unified execution infrastructure for stateful agentic EDA," arXiv:2603.25243, 2026.

[11] Y. Lu, S. Liu, Q. Zhang, and Z. Xie, "RTLLM: An open-source benchmark for design RTL generation with large language model," in *Proc. ASP-DAC*, 2024.

[12] J. Ma, J. Wang, Y. Ma, Z. Zhuang, and X. Liu, "NetlistBench: Evaluating LLM reliability in SPICE netlist recognition and manipulation," arXiv:2608.12197, 2026.

[13] F. Liparulo and F. Pierri, "Benchmarking AI agents for hardware design automation via MCP tool calling," arXiv:2608.26199, 2026.

[14] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, "Gorilla: Large language model connected with massive APIs," in *Proc. NeurIPS*, 2024.

[15] Y. Qin *et al.*, "ToolLLM: Facilitating large language models to master 16000+ real-world APIs," in *Proc. ICLR*, 2024.

[16] S. G. Patil *et al.*, "The Berkeley Function Calling Leaderboard (BFCL): From tool use to agentic evaluation of large language models," in *Proc. ICML*, 2025.

[17] S. Kambhampati *et al.*, "Position: LLMs can't plan, but can help planning in LLM-Modulo frameworks," in *Proc. ICML*, 2024.

[18] F. Batole, F. Khomh, and H. Rajan, "RefineAct: Automatic runtime verification of LLM agent actions," in *Proc. IEEE/ACM ASE*, 2026.

[19] Y. Liu *et al.*, "ToolGate: Contract-grounded and verified tool execution for LLMs," in *Findings of ACL*, 2026, pp. 9653–9684.

[20] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, "τ-bench: A benchmark for tool-agent-user interaction in real-world domains," in *Proc. ICLR*, 2025.

[21] "Beyond equivalence: Checking task completion in LLM-based netlist analysis, editing, and optimization," concurrent submission, 2026.

[22] R. Brayton and A. Mishchenko, "ABC: An academic industrial-strength verification tool," in *Proc. CAV*, LNCS 6174, 2010, pp. 24–40.